CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 4: Singly Linked List: Dynamic Insertion, Deletion, and Traversal

Practical Aim: To implement core singly linked list operations and develop the ability to manipulate dynamic sequences through real-world problem scenarios.

Problem 1 :

A hospital manages a queue of patient tokens. New critical patients must be added to the front, routine patients are added to the end, and occasionally a patient with a priority number must be inserted at a specific position in the queue. Given a sequence of such operations, implement all three insertion types and print the final queue after each operation.

For Code:

https://github.com/ayushposhiya00-lab/FDSA/tree/main/DSA/Practical-4

Output:



Problem: The hospital queue needs three types of insertion: at the front for critical patients, at the end for routine patients, and at a specific position for priority patients. The program must also print the queue after every insertion. If the given position is greater than the queue length, it should be handled properly.

Solution: Use linked-list insertion at the required position by traversing the queue. If the position is invalid, insert at the end or display an error.

Problem 2 :

The hospital also needs to remove a patient token from the queue when a patient leaves, print all remaining tokens from last to first for an end-of-day audit, and display the full queue from front to back at any point. Implement deletion by value, reverse printing, and forward traversal on the same queue from Problem 4.1a.

For Code:

https://github.com/ayushposhiya00-lab/FDSA/tree/main/DSA/Practical-4

Output:



Problem: The hospital needs to remove a patient by token value and display the remaining queue. It must support both normal front-to-back printing and reverse last-to-first printing without changing the original queue.

Solution: Traverse the linked list to find and delete the required value. Use normal traversal for forward printing and recursion for reverse printing without extra storage.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation:

Problem 1: Inserting at a position beyond the current length of the list is an invalid operation. Should your solution silently insert at the end, report an error, or do something else entirely? Justify your choice and explain what assumption about the caller your choice makes.

Problem 2: If the node to be deleted is the only node in the list, how many pointers need to change and what must they be set to? How is this case structurally different from deleting a node in the middle?

ANSWERS:

ANS-1:→Inserting a node at a position beyond the current length of the list should be treated as an invalid operation and an error should be reported. The program should not silently insert the node at the end because that would change the meaning of the requested position.

For example, if the list has 5 nodes and the user asks to insert at position 8, position 8 does not exist. The program should display an error such as "Invalid position" and leave the list unchanged**.**

ANS-2:→ If the node is the only node in the list, only the head pointer changes and is set to NULL; unlike deleting a middle node, there is no previous or next node whose link needs to be changed.